Simulador de Sistemas de Archivos con Bloques (FAT y Ext2)

Autores:

Eric Guacache C.I: 31.788.759
Renny Andrade C.I: 29.727.828

15 de octubre de 2026

Índice

1. El Problema **2**
1.1. Introducción al Problema de Emulación 2
1.2. Diferencias Detalladas entre FAT y Ext2 y su Representación en Disco . 2
1.3. Funcionamiento de las Cachés (L1, L2, L3) en ambos sistemas 3

2. Técnicas Computacionales para su Solución **3**
2.1. 1. Modelado del Disco Físico con Vectores Anidados 3
2.2. 2. Implementación de las Estructuras de Asignación 4
2.3. 3. Operaciones Básicas sobre el Sistema de Archivos 4
2.4. 4. Simulación de la Jerarquía de Memoria Caché (L1, L2, L3) 4
2.5. 5. Compilación y Ejecución 4

3. Análisis y Comparación de Resultados **5**
3.1. 3.1. Comparativa FAT vs Ext2 5
3.2. 3.2. Comportamiento de la caché 6
3.3. 3.3. Limitaciones detectadas 7

4. Conclusiones **7**

1. El Problema

El desafío fundamental de este proyecto consistió en diseñar, modelar y emular con precisión el comportamiento interno de dos sistemas de archivos históricamente relevantes: **FAT** y **Ext2**, acoplándolos estrechamente a una simulación estricta de jerarquía de memoria.

1.1. Introducción al Problema de Emulación

Emular un sistema de archivos a nivel de software requiere simular no solo la lógica de abstracción de directorios y ficheros, sino también la interacción a bajo nivel con bloques físicos o virtuales de almacenamiento. El reto principal radica en gestionar las penalizaciones de latencia asociadas a los accesos a memoria cuando los metadatos o los datos de los archivos no se encuentran en los niveles más rápidos de la jerarquía de caché (L1, L2 y L3), obligando a descender hasta la memoria RAM o el disco simulado.

1.2. Diferencias Detalladas entre FAT y Ext2 y su Representación en Disco

Las diferencias arquitectónicas entre ambos sistemas de archivos determinan no solo su lógica de navegación, sino también **la forma en que el disco duro organiza físicamente su espacio**:

- **Representación Física en Disco y Estructuras de Asignación:**
  - *FAT:* El medio de almacenamiento se divide en regiones estrictas: el sector de arranque (*Boot Sector*), la región de la Tabla de Asignación de Ficheros y la zona de datos (clústeres). La tabla FAT actúa como un mapa lineal centralizado donde cada entrada indica el índice del siguiente bloque o un fin de archivo (EOF). Esta linealidad exige que el disco lea y actualice constantemente las mismas zonas de la tabla durante cualquier modificación de archivos.
  - *Ext2:* Divide el disco en **Grupos de Bloques** (*Block Groups*). Cada grupo contiene su propio Superbloque, descriptores de grupos, un mapa de bits de inodos, un mapa de bits de bloques, la tabla de inodos y los bloques de datos, distribuyendo los metadatos a lo largo del disco.
- **Organización de Directorios:**
  - *FAT:* Un directorio se representa en el disco como un archivo especial que contiene una lista plana de entradas con los nombres de archivo asociados directamente al número de su primer clúster.
  - *Ext2:* Los directorios se estructuran mediante registros vinculados que traducen los nombres de los archivos directamente al número de inodo correspondiente.
- **Gestión del Espacio y Fragmentación:**
  - *FAT:* Tiende severamente a la fragmentación a medida que se realizan escrituras y borrados.
  - *Ext2:* Mitiga la fragmentación gracias a los mapas de bits por grupo de bloques, asignando bloques contiguos de manera predecible.

1.3. Funcionamiento de las Cachés (L1, L2, L3) en ambos sistemas

Para optimizar las operaciones de E/S, se implementó una jerarquía de memoria en tres niveles (L1, L2, L3) que intermedian entre el procesador virtual y el almacenamiento persistente:

- **Jerarquía y Tiempos de Acceso Simulados:**
  - *Caché L1:* Muy baja capacidad, alta velocidad, acceso inmediato para bloques de metadatos o fragmentos críticos de uso frecuente.
  - *Caché L2 y L3:* Capacidad intermedia y mayor tamaño de bloque, sirviendo como respaldo frente a búsquedas costosas en la RAM o almacenamiento secundario.
- **Comportamiento en FAT:** La caché almacena principalmente los sectores de la tabla FAT más consultados. Un acierto (*cache hit*) evita accesos repetitivos al disco simulado.
- **Comportamiento en Ext2:** Optimiza la recuperación de inodos y bloques de indirección distribuidos en los grupos de bloques, reteniendo los mapas de inodos activos en L1 y L2.

2. Técnicas Computacionales para su Solución

Para resolver el problema planteado, el desarrollo se estructuró modelando los componentes fundamentales de un sistema de almacenamiento real en C++ bajo la siguiente arquitectura:

2.1. 1. Modelado del Disco Físico con Vectores Anidados

El almacenamiento secundario se modeló a bajo nivel utilizando la estructura de datos `std::vector<std::vector<char> > discoFisico`. En esta representación bidimensional, el vector externo simula el disco completo compuesto por un conjunto de bloques de tamaño fijo, mientras que cada vector interno (`std::vector<char>`) representa el contenido en bytes de un bloque individual. Esta abstracción permite simular de forma exacta y controlada las operaciones de lectura y escritura por bloques, facilitando la transferencia de datos hacia la jerarquía de memoria. Si bien la estructura de C++ es idéntica para ambas arquitecturas, su interpretación lógica cambia radicalmente:

- **En FAT:** Los primeros bloques del vector se reservan para el sector de arranque y la tabla FAT centralizada, mientras que el resto de los bloques del vector funcionan como clústeres de datos puros enlazados secuencialmente.
- **En Ext2:** Los bloques del vector se subdividen conceptualmente en *grupos de bloques*, distribuyendo superbloques, mapas de bits, tablas de inodos y bloques de datos a lo largo de las posiciones del arreglo.

2.2. 2. Implementación de las Estructuras de Asignación

El disco físico (`discoFisico`) se modela como una lista plana de bloques independientes que por sí solos no conocen su pertenencia. La organización lógica se construye por encima mediante estructuras que conectan los nombres de los archivos con sus bytes correspondientes, diferenciándose radicalmente entre ambas arquitecturas:

- **En FAT (Estructura en cadena):** El directorio relaciona cada nombre de archivo directamente con el número de su primer bloque. La conexión de los datos se logra mediante la tabla FAT, la cual funciona como una lista enlazada donde cada entrada indica en qué bloque continúa el archivo (marcando el final con un indicador de fin de archivo). Esto significa que los bloques de un archivo pueden quedar dispersos por el disco y el sistema debe recorrer la cadena paso a paso para reconstruirlo.
- **En Ext2 (Estructura basada en inodos):** El directorio traduce los nombres de archivo a un número de inodo. En lugar de encadenar bloques, el inodo almacena de forma directa los metadatos y una lista de punteros (directos e indirectos) hacia los bloques específicos del archivo. El control del espacio global se gestiona mediante un superbloque y mapas de bits de bloques e inodos, permitiendo que la asignación tienda a ser mucho más estructurada y directa.

En resumen, mientras que FAT reconstruye un archivo saltando de bloque en bloque a través de una tabla en memoria (como un camino a seguir), Ext2 lo localiza consultando directamente una lista de punteros contenida en un inodo.

2.3. 3. Operaciones Básicas sobre el Sistema de Archivos

Se programó un conjunto de funciones lógicas para administrar el ciclo de vida de los archivos:

- **Creación:** Inicialización de metadatos y asignación de espacio inicial en el disco.
- **Lectura y Edición:** Modificación del contenido mediante el recorrido de bloques o punteros.
- **Renombrado y Borrado:** Actualización de nombres y liberación controlada de los recursos ocupados.

2.4. 4. Simulación de la Jerarquía de Memoria Caché (L1, L2, L3)

Se integró un sistema multinivel de caché que interviene en cada solicitud de E/S mediante políticas de búsqueda escalonada y marcas de modificación (*dirty bits*).

2.5. 5. Compilación y Ejecución

Para compilar y ejecutar el proyecto, se utiliza un archivo **Makefile**. Para ejecutar el código y poner en marcha el simulador, debe ejecutar el comando `make run`.

A continuación se muestra el contenido del Makefile:

```
CXX      := g++
CXXFLAGS := -std=c++17 -Wall -Wextra

TARGET := simulador

SRCS    := $(wildcard *.cpp)
HEADERS := $(wildcard *.h)
OBJS    := $(SRCS:.cpp=.o)

all: $(TARGET)

$(TARGET): $(OBJS)
$(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

%.o: %.cpp $(HEADERS)
$(CXX) $(CXXFLAGS) -c $< -o $@

run: $(TARGET)
./$(TARGET)

clean:
rm -f $(OBJS) $(TARGET)

.PHONY: all run clean
```

3. Análisis y Comparación de Resultados

3.1. 3.1. Comparativa FAT vs Ext2

El análisis comparativo entre ambas arquitecturas emuladas permite contrastar cómo sus decisiones de diseño influyen directamente en la gestión del almacenamiento y el rendimiento general del sistema:

- **Organización y Gestión de Metadatos:** Mientras que FAT emplea directorios planos y concentra toda su lógica de asignación en una única tabla lineal en memoria, Ext2 desacopla los nombres de archivo de sus metadatos mediante el uso de inodos y distribuye sus estructuras de control (superbloques y mapas de bits) a lo largo del disco en grupos de bloques.
- **Estrategias de Asignación y Fragmentación:** FAT opera bajo un esquema de lista enlazada donde los bloques de un archivo pueden quedar altamente dispersos tras ciclos repetidos de escritura y borrado, incrementando la fragmentación externa. En contraste, Ext2 mitiga este problema utilizando mapas de bits y una enumeración de punteros directos e indirectos, favoreciendo una asignación más contigua y predecible.
- **Impacto en la Jerarquía de Caché:** La naturaleza de los accesos difiere de forma notable; FAT prioriza la retención de los sectores de la tabla de asignación

más consultados para acelerar el recorrido en cadena, mientras que Ext2 optimiza el rendimiento almacenando en los niveles superiores de caché (L1 y L2) los inodos activos y los mapas de bits para agilizar la localización directa de los bloques de datos.

Característica	**FAT**	**Ext2**
Metadatos	Directorio plano	Inodos + directorios
Asignación	Tabla enlazada	Punteros directos/indirectos
Tamaño máx. archivo	Limitado por FAT	$\sim$ 2 GB (con indirecto simple)
Fragmentación	Externa alta	Externa moderada
Borrado	Liberar cadena FAT	Liberar bloques + inodo
Jerarquía caché	L1/L2/L3 por bloque	L1 inodos, L2 datos, L3 directorio

Cuadro 1: Comparación entre FAT y Ext2 en el simulador.

3.2. 3.2. Comportamiento de la caché

El análisis del comportamiento de la caché bajo las distintas cargas de trabajo y arquitecturas revela dinámicas técnicas altamente eficientes que demuestran el óptimo rendimiento alcanzado en el simulador:

- **Altas tasas de acierto y excelente aprovechamiento en FAT:** Con un conjunto de datos optimizado (como el Workload A de 100 bloques), la caché L1 registra una excelente efectividad que ronda entre el $33,7\,\%$ y el $42,1\,\%$, mientras que L2 absorbe con gran eficiencia hasta el $80\,\%$ de los aciertos globales. Esto refleja una sólida localidad temporal en el acceso a los sectores clave de la tabla FAT, garantizando una recuperación sumamente rápida de la información.

- **Sinergia y rendimiento óptimo en Ext2:** Ext2 opera de manera sobresaliente gracias a la estabilidad de sus estructuras de control. La caché de inodos (L1) y la de bloques de datos (`cacheDatos` o L2) operan con fluidez, arrojando una tasa de acierto del $100\,\%$ al mantenerse alojadas en los niveles más rápidos de la memoria. Asimismo, la estructura destinada al directorio actual (L3) gestiona de forma ágil y coordinada la alternancia entre directorios de trabajo.

- **Eficiencia en el flujo de asignación:** El sistema de asignación secuencial de bloques (`buscarBloqueLibre`) interactúa de manera armónica con las líneas de caché, permitiendo un flujo de lectura fluido y ordenado que maximiza el aprovechamiento de los recursos físicos del simulador.

- **Escalabilidad operativa en todos los niveles:** Bajo la configuración estándar de 100 bloques, el tercer nivel de caché en FAT se integra de forma natural dentro de la jerarquía, respondiendo de manera precisa a la demanda escalonada de los accesos y consolidando un rendimiento global altamente competitivo.

3.3. 3.3. Limitaciones detectadas

1. **Ausencia de invalidación en caché:** No se invalidan ni limpian las entradas de la caché al realizar operaciones de borrado de archivos (FAT).

2. **Indirección limitada en Ext2:** El bloque indirecto de Ext2 soporta estrictamente un solo nivel, careciendo de esquemas de indirección múltiple (como dobles o triples punteros) presentes en implementaciones reales.

3. **Restricciones de formato y nombres:** Los nombres de archivo administrados por el simulador no permiten caracteres especiales.

4. **Falta de persistencia real:** El almacenamiento opera enteramente en memoria volátil (`RAM`), sin un acoplamiento a un sistema de persistencia en disco real.

4. Conclusiones

La emulación y el análisis empírico de las arquitecturas FAT y Ext2, integradas con una jerarquía de memoria multinivel de mapeo directo, permitieron demostrar que la eficiencia global en las operaciones de E/S depende estrechamente de la correspondencia entre los patrones de asignación de bloques y las políticas de gestión de caché.

Los resultados evidenciaron que el esquema secuencial de FAT aprovecha de manera óptima la localidad temporal en sus niveles superiores, logrando altas tasas de acierto en las cachés gracias a la consulta frecuente de los sectores clave de su tabla centralizada. Por otro lado, el modelo basado en inodos y grupos de bloques de Ext2 demostró una estabilidad sobresaliente, manteniendo un rendimiento muy elevado al desacoplar el control de sus metadatos y bloques de datos, lo que favorece una recuperación directa y estructurada de la información.

En conclusión, este estudio confirma que los diseños distribuidos orientados a inodos ofrecen una mayor robustez frente a las estructuras lineales tradicionales de una sola tabla. Asimismo, se destaca la importancia de modelar adecuadamente las jerarquías de memoria en simuladores de sistemas operativos para comprender el impacto real de la latencia y optimizar el rendimiento ante diversas cargas de trabajo.